# KEVIN STEFANOV
# CPU Microarchitecture Performance Analysis 1

September 2026

*How code interleaving led to higher instruction-level parallelism,*
*resulting in a 10% latency boost for an already well-optimized C function,*
*explained from a CPU architecture standpoint using performance profiling tools.*

## Abstract

This writeup presents the result of a detailed low-level performance experiment I recently did.

I had a C function with 2 call sites back to back. The function was already well optimized using GCC intrinsics to emit specific assembly instructions. I concluded it needed further optimizing after a flamegraph showed these 2 call sites were where the running system spends most of its time.

After Linux Perf kept showing ~10% Core Bound and Port Utilization TMA metrics and that 61% of CPU cycles in which only 1 micro-op execution port was utilized were attributed to that function, I decided to attempt to interleave the code for these 2 call sites in hopes to expose more instruction-level parallelism and provide the CPU pipeline with an additional sequence of independent assembly instructions it can work on, while stuck on dependency chains in the instructions of the first call site.

**The experiment was a success.**
Average total wall clock time of the 2 function calls went from 4703 down to 4213 microseconds.

In this report, I look at various aspects of how the function runs on the CPU, given by Top-down microarchitecture analysis methodology, Linux Perf and LLVM Machine Code Analyzer. I put it all together as evidence that the latency optimization was achieved thanks to higher instruction-level parallelism, which enabled the CPU micro-op execution ports to sit idle in fewer cycles overall.

## Background

Modern x86_64 CPUs break down assembly instructions into smaller units of work called micro operations (µOps). The work represented by them is carried out by a small number of hardware structures called micro-op execution ports, residing in the CPU back-end. The CPU architecture is such that several execution ports can be fed a new micro-op in the same CPU cycle and they all operate independently, meaning **they can be utilized in parallel**. The architecture also allows instructions to be **executed out-of-order** for boosted performance, while still being "officially retired" in correct program order by using a hardware structure called the Reorder Buffer.

This is called **instruction-level parallelism** (ILP) and achieving a high degree of it leads to more optimized code that runs faster. ILP can be hindered by factors such as sub-optimal memory access patterns that lead to data not being present in the CPU caches when its instruction needs it, and by dependency chains in the incoming assembly instructions, where instruction X+N must wait for one that came before it, instruction X, to finish and produce its result so that X+N can use that result in its own operands. This degrades the parallel execution of several micro-ops by several execution ports at once. One remedy for this is interleaving independent work in code where such dependencies are present, so the CPU pipeline can find more out-of-order instructions to work on.

## Introduction

In my Rosetta Secret Communications system, the running relay server is constantly being polled every 100 milliseconds by each connected client. Each time, the server must validate that client's signature, which is what contains the 2 calls to the function in question - Montgomery Modular Powering, which in turn spends most of its time computing Montgomery Modular Multiplications.

A flamegraph showed these 2 calls to be where the majority of the server's time is being spent:

```
sudo perf record -e cpu_core/cycles/P -g - bin/rosetta-server
sudo perf script | stackcollapse-perf.pl | flamegraph.pl > my-flamegraph.svg
```

I used the Perf tool to obtain two other important measurements:

```
sudo perf stat --topdown -- bin/rosetta-server
sudo perf stat -M tma_core_bound,tma_memory_bound -- bin/rosetta-server
sudo perf stat -M tma_ports_utilization - bin/rosetta-server
```

This revealed the Core Bound and Ports Utilization sub-buckets of the Backend Bound bucket in Top-down microarchitecture analysis methodology (TMA) to be consistently reported as ~**10%**.

Then, I used the following commands:

```
sudo perf record -e cpu_core/exe_activity.1_ports_util/ -g - bin/rosetta-server
sudo perf report
```

They revealed that **61%** of all CPU cycles in which only 1 micro-op execution port was being utilized were attributed exactly to my Montgomery Modular Multiplication function. Using clock_gettime() around the various parts that make up the function, I concluded that the majority of one call's time is spent in **the following inner loop** (slightly rewritten to be more compact for this illustration):

```
static inline __attribute__((always_inline)) void mont_mul_inner_loop
(      uint64_t* const __restrict__ R_limbs, const uint64_t* const __restrict__ N_limbs,
 const uint64_t* const __restrict__ X_limbs, const uint64_t* const __restrict__ Y_limbs,
 const uint64_t i, const unsigned long long q)
{
    uint8_t C, D;
    unsigned long long Ul, Uh, Vl, Vh, W;

    /* T, a 3-limb variable, resides in R->bits buffer for optimization. */
    #define T  ( (unsigned long long*)(R_limbs + (MONT_L + 1)) )
    for(u64 j = 1; j < MONT_L; ++j)
    {
        Ul   = _mulx_u64(q, N_limbs[j], &Uh);
        Vl   = _mulx_u64(Y_limbs[i], X_limbs[j], &Vh);
        C    = _addcarryx_u64((u8)0, Ul, R_limbs[j], &Ul);
        Uh  += (u64)C;
        D    = _addcarryx_u64((u8)0, Vl, T[1], &Vl);
        C    = _addcarryx_u64((u8)0, Ul, Vl, T);
        D    = _addcarryx_u64(D, Uh, Vh, &W);
        C    = _addcarryx_u64(C, W, T[2], T + 1);
        T[2] = (u64)C + (u64)D;
        R_limbs[j-1] = *T;
    }
    #undef T
}
```

**Interleaved version of the loop**, which achieved the 10% speedup this write-up tries to explain:

```
static inline __attribute__((always_inline))
void dual_mont_mul_inner_loop(      uint64_t* const __restrict__ R_limbs,
                                    uint64_t* const __restrict__ R2_limbs,
                                const uint64_t* const __restrict__ N_limbs,
                                const uint64_t* const __restrict__ X_limbs,
                                const uint64_t* const __restrict__ X2_limbs,
                                const uint64_t* const __restrict__ Y_limbs,
                                const uint64_t* const __restrict__ Y2_limbs,
                                const uint64_t i, const unsigned long long q,
                                const unsigned long long q2)
{
    uint8_t C, D;
    unsigned long long  Ul, Uh, Vl, Vh, W;

    /* T, a 3-limb variable, resides in R->bits buffer for optimization */
    #define T  ( (unsigned long long*)(R_limbs  + (MONT_L + 1)) )
    #define T2 ( (unsigned long long*)(R2_limbs + (MONT_L + 1)) )

    for(u64 j = 1; j < MONT_L; ++j)
    {
        /* Interleaved. One iteration for call 1's arguments. */
        Ul    = _mulx_u64(q, N_limbs[j], &Uh);
        Vl    = _mulx_u64(Y_limbs[i], X_limbs[j], &Vh);
        C     = _addcarryx_u64((u8)0, Ul, R_limbs[j], &Ul);
        Uh   += (u64)C;
        D     = _addcarryx_u64((u8)0, Vl, T[1], &Vl);
        C     = _addcarryx_u64((u8)0, Ul, Vl, T);
        D     = _addcarryx_u64(D, Uh, Vh, &W);
        C     = _addcarryx_u64(C, W, T[2], T + 1);
        T[2]  = (u64)C + (u64)D;
        R_limbs[j-1] = *T;

        /* Interleaved. One iteration for call 2's arguments. */
        Ul    = _mulx_u64(q2, N_limbs[j], &Uh);
        Vl    = _mulx_u64(Y2_limbs[i], X2_limbs[j], &Vh);
        C     = _addcarryx_u64((u8)0, Ul, R2_limbs[j], &Ul);
        Uh   += (u64)C;
        D     = _addcarryx_u64((u8)0, Vl, T2[1], &Vl);
        C     = _addcarryx_u64((u8)0, Ul, Vl, T2);
        D     = _addcarryx_u64(D, Uh, Vh, &W);
        C     = _addcarryx_u64(C, W, T2[2], T2 + 1);
        T2[2] = (u64)C + (u64)D;
        R2_limbs[j-1] = *T2;
    }
    #undef T
    #undef T2
}
```

The interleaving was very simple. It merely copies the body of the loop and pastes it right after, so that in one iteration of the loop, as far as the source code is concerned, we actually carry out 1 iteration for **function call 1's arguments** and 1 iteration for **function call 2's arguments**. Thus, the out-of-order CPU pipeline is now presented with an entire new independent stream of assembly instructions it can pick from and work on, while waiting for dependencies of the other stream of instructions to be resolved, thus increasing instruction-level parallelism.
The reason N is still passed only once, instead of having N1 and N2, is that the Montgomery Modulus N is the same for both function calls whose inner loops were interleaved here.

For the cryptography parameters my system runs with (3072-bit public keys, 320-bit private keys), each signature validation runs this loop for **a total of 2,165,760 iterations**.

This profiling was done on my laptop running native up-to-date Fedora Linux which has a 13th Generation Intel Core i7 CPU, whose architecture is Raptor Cove, very similar to Golden Cove.

Kernel 6.19, relay server compiled by GCC 15.2.1 with -O3 -march=native. Efficiency cores were not used, only regular Performance Cores were what the server ran on. Isolating the process to run on specific P-cores using the taskset command had no effect on measured latency.

Looking closely at the code for the original inner loop, we spot **a clear dependency**:

```
D = _addcarryx_u64((u8)0, Vl, T[1], &Vl);
C = _addcarryx_u64((u8)0, Ul, Vl, T);
D = _addcarryx_u64(D, Uh, Vh, &W);
C = _addcarryx_u64(C, W, T[2], T + 1);
```

The first call stores its result in `Vl` and a produced carry in D. The next call immediately uses `Vl` as input. The 4th call uses results produced by the 2nd and 3rd calls (64-bit variable W and the carry C).

The CPU will not be able to execute the assembly instructions emitted for this C code in parallel, which I theorized is a significant contributing factor to the 61% number we got. Recall, this is the percentage of all CPU cycles with underutilized execution ports **attributed to this function** alone.

Thus, I came up with the idea of writing this new function that simply interleaves the C code for the inner loop, as shown on the previous page.

My hope was that this would enable the CPU to replace a good portion of the wasted cycles with underutilized execution ports with cycles where the pipeline was able to **find independent out of order instructions from the second function** call's instruction stream. Indeed, the measurements I present in the next sections of the writeup strongly point to me having achieved exactly that.

## Measuring Latency

`clock_gettime(CLOCK_MONOTONIC_RAW, …)` is implemented on modern Linux as a very efficient memory read from a field containing the current time, which the kernel maps for each process, without having to do a system call or a context switch at all. It is flawless for wall clock time measurements for our purpose, accurate to the microsecond, which is why I have used it here. I simply surround the code I'm timing with 2 such calls and take the time difference.

The code being timed is, of course, the 2 Montgomery Modular Powering function calls back to back, or the 1 interleaved function call in the optimized case. Tens of thousands of latency readings were obtained per measurement and a consistent average latency was successfully observed.

Once we collect a batch of 500 latency readings, a stabilized average latency is computed from the 500 readings. Multiple stabilized averages then produce the final observed average latency.

Here is the procedure by which I observed consistent average latency across measurements.

First, eliminate outliers from each batch of 500 latency readings:

```
1. Sort the latency readings in ascending order. Split into 2 halves.
2. Middle-indexed reading of first half (median) = Q1. The 25th percentile.
3. Middle-indexed reading of other half (median) = Q3. The 75th percentile.
4. Interquartile range IQR = Q3 - Q1.
5. LowerFence LF = Q1 - (1.5 * IQR).
6. UpperFence UF = Q3 + (1.5 * IQR).
7. Outliers are all readings LESS THAN LowerFence OR GREATER THAN UpperFence.
```

With the outliers gone, we take the average of the remaining latency readings.

Repeat this process of collecting batches of 500 readings, removing outliers and noting their average 128 times. This gives tens of thousands of cleaned latency readings to work with. Using the 128 stabilized averages, we compute a final observed average latency in a given measurement.

Using this method, I consistently get final averages within 2 microseconds of the median average. This is solid, given that each latency reading is thousands of microseconds.

All in all, I have made sure to have enough individual readings, get them cleaned up from outliers, and that the mechanism used to take each latency reading is itself fast enough that it won't interfere with the profiling process in any meaningful way.

## Hypothesis for why the optimization worked, backed by concrete evidence

To state my hypothesis clearly:

The non-interleaved version had some number of CPU cycles during which the dependency chain caused micro-ops to sit in the Reservation Station instead of being fed to the micro-op execution ports, starving the execution ports of work and keeping them idle, producing wasted cycles.

In the interleaved version, the second function call's stream of assembly instructions caused part of these wasted cycles **to not be wasted anymore** because now the CPU is able to find independent out-of-order instructions from that second function call and feed the micro-op execution ports with useful work from them **during cycles that would have otherwise been wasted** waiting on dependencies for the 1st function call's instructions to be resolved.

This would mean that the interleaved version exhibits better instruction-level parallelism.

I present **5 pieces of evidence** that back my hypothesis up to varying degrees. I collected them using Linux Perf, LLVM Machine Code Analyzer & by inspecting compiler-emitted assembly code.

Evidence Item 1: Reduced blame for the total cycles with under-utilized execution ports.

Recall, by running the following commands, we obtain the percentage of all CPU cycles where only 1 execution port is being utilized, attributed to each function that contributed to the wasted cycles.

```
sudo perf record -e cpu_core/exe_activity.1_ports_util/ -g -- bin/rosetta-server
sudo perf report
```

Recall further, Montgomery Modular Multiplication's blame for such cycles was originally 61%.

With the interleaved version, that number dropped to 55%. While not a gargantuan reduction, we have to remember Linux Perf measures the whole running relay server. What this change tells us is that a non-trivial part of the blame this function gets for ALL cycles with under-utilized execution ports **was moved to a different part of the system**. The 2 function calls are now a smaller contributor to wasted CPU cycles with under-utilized execution ports. This directly supports my hypothesis that higher ILP, thanks to the 2nd function call's stream of assembly instructions, allowed the Reservation Station to find more independent out-of-order micro-ops for the execution ports to work on, **where the initial non-interleaved version was wasting CPU cycles**.

In conclusion, I can say that this supports my hypothesis. Evidence grade: Strong.

Evidence items 2, 3 and 5 come from using **LLVM Machine Code Analyzer.**

We must keep in mind that it is a static simulation of how a particular CPU microarchitecture will handle a given piece of assembly code, it's not showing us what actually happened in reality. This has to be taken into account when assessing the strength of evidence gathered from it. Further, Raptor Lake is not yet implemented in the simulator at this time, however, the only difference with Tiger Lake is “a better branch predictor”, no other details in the microarchitecture have changed, they're nearly identical, so running against Tiger Lake should still provide useful results.

Evidence Item 2: Overall larger execution port utilization percentages.

| µ0p Execution Port Number | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| % Utilization: Original | 58.7 | 31.5 | 33.1 | 33.1 | 49.5 | 31.5 | 62.9 | 49.5 | 49.5 | 49.5 | 33.1 | 31.5 |
| % Utilization: Interleaved | 55.5 | 34.7 | 41.7 | 41.7 | 50.9 | 34.7 | 60.3 | 50.9 | 50.9 | 50.9 | 41.7 | 34.7 |
| Utilization Difference | −3.2 | +3.2 | +8.6 | +8.6 | +1.4 | +3.2 | −2.6 | +1.4 | +1.4 | +1.4 | +8.6 | +3.2 |

These numbers were obtained by plugging the compiler-generated assembly code for the inner loop in LLVM-MCA. Since the inner loop represents 99% of the work done and time spent, it gives us a good idea of how execution port utilization percentages change between the two versions.

Some have decreased, but putting all the differences together, we see a net positive overall. If we didn't know that the interleaved version was faster, we might have raised an eyebrow and asked whether the ports with increased utilization are actually doing any useful work, or are simply working through extra memory moves due to new register pressure, for example. Though, given that the interleaved version with higher overall % execution port utilization **is faster**, this becomes much less of a concern. It hasn't disappeared, but we can somewhat confidently say that this piece of evidence is heavily pointing in favor of my hypothesis, with further analysis zooming in on what each port is actually doing in both versions needed, before we can claim this with 100% certainty.

In conclusion, I can say that this likely supports my hypothesis, with more detailed analysis focusing on each execution port required before being absolutely certain.
Evidence grade: **Points toward support for my hypothesis, inconclusive without deeper analysis.**

The third piece of evidence comes by looking at LLVM MCA's **timeline view**. This is a diagram that shows precisely, cycle by cycle, what stage **each incoming assembly instruction** will be in, during its entire residence in the CPU Backend. From dispatched, to waiting in Reservation Station, to being executed, to finished but waiting in Reorder Buffer, to the cycle it's retired in.

Evidence Item 3: Higher number of assembly instructions residing in the CPU backend in parallel

Let's look at the following illustrations I extracted from LLVM-MCA's Timeline View.

They show the first and last instruction that are residing together in parallel **anywhere within the CPU backend**. This is after the initial warmup period of the loop, after the initial CPU cycles, once the loop's architectural runtime behaviour settles down. The “R” of the first instruction and the “D” of the last instruction are in the same CPU cycle. This is the cycle in which the oldest instruction that had entered the CPU backend now leaves it – (R)etired, and the newest instruction in parallel with it enters the CPU backend, (D)ispatched to the Reservation Station either from the Fetch-Decode route or from the µ0p cache, which is faster and holds µ0ps for repeated execution.

**How to read the two illustrations:**
Each character from D to R is one CPU cycle. Time in cycles moves left to right.
**D**: Instruction was dispatched to the Reservation Station by the CPU Frontend at this cycle.
**=**: Instruction waits in Reservation Station, can't yet go to its required execution port(s).
**e**: Instruction's micro-ops are being worked on by execution port(s).
**E**: Instruction's result is ready and visible to the rest of the CPU pipeline at this cycle.
−: Instruction's micro-ops are sitting in the Reorder Buffer, waiting to be correctly retired in-order.
**R**: Instruction is retired in correct program order from the Reorder Buffer at this cycle.

Non-interleaved version:

```
------------------------------------------------------------------------
D==============eeeeeeeeeeeeER  mov qword ptr [rax + 400], rcx
...parallel instructions...
                             D======E-----------------R  movzx  esi, sil
------------------------------------------------------------------------
```

In this displayed timeline excerpt:

First: Iteration 21, instruction 31
Last : iteration 24, instruction 17
Instructions per loop iteration: 40 (0 to 39).
Thus, **108 instructions** are being processed by the CPU backend in parallel after initial warmup.

Interleaved version:

```
------------------------------------------------------------------------
D==================eeeeeeeeeeeeER  mov qword ptr [rdx + 400], rsi
...parallel instructions...
                                  D=eE------------------------R  cmp rax, 48
------------------------------------------------------------------------
```

In this displayed timeline excerpt:

First: Iteration 9, instruction 30
Last : iteration 10, instruction 72
Instructions per loop iteration: 73 (0 to 72).
Thus, **117 instructions** are being processed by the CPU backend in parallel after initial warmup.

More instructions are residing **at any stage in the CPU backend in parallel** in the interleaved version. This is directly showing higher instruction-level parallelism and supports my hypothesis.
Evidence grade: **Strong**.

The last 2 evidence items are related to each other. One comes from LLVM Machine Code Analyzer, the other is a simple observation made by looking at the compiler-generated assembly code for the inner loops of the original and interleaved versions.

Evidence Item 4: Smaller total number of assembly instructions emitted for the inner loop.

If we inspect what assembly instructions GCC emitted for the inner loop, we find:

```
Non-interleaved: 40 instructions PER ITERATION, 2 work units = 80 instructions.
Interleaved  x2: 73 instructions PER ITERATION, 2 work units = 73 instructions.
```

A unit of work here is having processed one 64-bit batch (limb) of a Big Integer's bits in the Montgomery Modular Multiplication algorithm. This happens in one loop iteration in the original version. The interleaved version performs one iteration on the 1st operand set and one iteration on the 2nd operand set, all in one larger C loop iteration. Same amount of work completed in fewer assembly instructions. Definitely a win. Evidence grade: **Strong.**

The fifth and last piece of evidence supporting my hypothesis comes from LLVM-MCA again. It reports **2 assembly instructions having a latency of 0 cycles**. This means that the instruction will not be processed by the micro-op execution ports at all. Instead, e.g. a **mov** with latency 0 can be completed at the Register Renaming stage, by swapping physical register file pointers. There are other similar microarchitectural optimizations purely in hardware, outside the scope of this report.

Both versions have 2 instructions with latency 0. Assuming that having such instructions with latency 0 arrive in the incoming stream of assembly instructions **more often is better**, we recall that 2 iterations in the original version execute in 80 instructions, while 2 iterations (in one larger interleaved iteration) execute in 73 instructions. Both have 2 zero-latency instructions. However, since the interleaved version takes 73 instructions to do the same amount of work that the original takes 80 instructions to do, it means that **the interleaved version will be seeing zero-latency instructions arrive more often** in the stream of instructions for the inner loop.

This is not conclusive evidence, since it needs deeper analysis focusing on each such instruction, but it does point toward support for my hypothesis, since instructions are being executed via hardware shortcuts instead of in the execution ports **more often in the interleaved version**, meaning the execution **ports could be free for other useful work more often** than in the original version, contributing to the latency boost and indirectly to higher instruction-level parallelism. Evidence grade: **Points toward support for my hypothesis, inconclusive without deeper analysis.**

## Analysis conclusion & possible future optimizations

I have explained from a CPU microarchitecture standpoint why my code interleaving optimization worked and gathered multiple pieces of evidence that point toward my explanation being correct. From the presented evidence, three items definitely support my hypothesis and two appear to show support for it, but need deeper analysis for their support to be conclusively proven. Please note this is my first ever deep performance analysis of this kind, many more to come in the future.

A possible optimization that is worth investigating in the future is spilling to vector registers instead of the L1 cache spills that the compiler generates at present. This could put pressure on certain execution ports that are used for vector register to general purpose register transitions, resulting in a slowdown, but it could also lower latency: it's potentially faster than touching L1 cache for spills.